

APT MALWARE CLASSIFICATION BASED ON N-GRAM ANALYSIS

COURSE:

CIS*6530 CYBER THREAT INTELLIGENCE AND ADVERSARIAL
RISK ANALYSIS

SUBMITTED BY:

SUPRIM DEVKOTA
KHUSHBUKUMARI HEMALKUMAR PATEL

MARCH, 2026

Contents

1	Introduction	1
2	Related Theory	2
2.1	Malware Classification and Machine Learning	2
2.2	Opcode-Based Representation and N-Grams	2
2.3	Vectorization Techniques	2
2.4	Feature Selection and Normalization	3
2.5	Classification Algorithms	3
2.6	Cross-Validation	3
2.7	Class Imbalance Handling	3
2.8	Evaluation Metrics and Confusion Matrix	4
3	Experimental Setup	5
3.1	Setup	5
3.2	Execution Commands	6
4	Methodology	7
4.1	Dataset	7
4.2	Preprocessing pipeline	7
4.3	Preprocessed dataset	8
4.4	Classification	9
4.4.1	Feature Engineering	9
4.4.2	Classifier Models	9
4.4.3	1-gram vs 2-gram Experiments	9
4.4.4	Cross-validation Design	10
4.4.5	Class Imbalance Handling	10
5	Results & Analysis	11
5.1	Performance Metrics	11
5.2	Comparative Analysis between n-grams	15
6	Conclusion	17

7	Future Work	18
	References	18
	Appendices	20

1. Introduction

This report presents a multi-class malware classification study focused on attributing samples to Advanced Persistent Threat (APT) groups using opcode-level features extracted from executable files. It builds on the opcodes extracted in the previous submission and examines whether statistical patterns in opcode sequences, specifically **1-gram and 2-gram** frequency distributions, provide enough distinguishing information to accurately link malware to its originating APT group.

Three classification algorithms are evaluated: **Support Vector Machines (SVM)**, **K-Nearest Neighbors** and **Decision Tree**. All experiments are conducted using **5-fold stratified cross-validation**, with performance assessed through **Accuracy, Precision, Recall, F-measure, and Confusion Matrix analysis**.

2. Related Theory

This section presents the key concepts and techniques used in the malware classification pipeline. The focus is on understanding how opcode-based features are transformed and utilized by machine learning models to classify malware samples into Advanced Persistent Threat (APT) groups.

2.1 Malware Classification and Machine Learning

Malware classification is the task of assigning a malicious software sample to a known category, such as an Advanced Persistent Threat (APT) group or malware family. In this work the problem is framed as a multi-class supervised learning task, in which each sample is associated with a single predefined label and the model learns discriminative patterns from a labelled training set [1].

2.2 Opcode-Based Representation and N-Grams

Opcodes are low-level instructions extracted from executable binaries that describe the operational behaviour of a program. Because malware tends to follow recognisable execution patterns, opcode sequences provide a compact, static representation of program functionality. Treating these sequences as text enables the application of Natural Language Processing (NLP) feature-extraction methods [2].

An n -gram is a contiguous sequence of n items drawn from a longer sequence. Applied to opcodes, a unigram (1-gram) represents a single instruction, while a bigram (2-gram) represents a pair of consecutive instructions. Larger n captures richer contextual dependencies at the cost of a substantially higher-dimensional feature space [2].

2.3 Vectorization Techniques

Machine learning models require numerical input. Vectorization transforms a sequence of tokens into a fixed-length numeric vector.

Count vectorization records the raw frequency of each token within a document, yielding a term-frequency vector.

TF-IDF (Term Frequency–Inverse Document Frequency) scales each term’s frequency by the logarithm of the inverse proportion of documents containing that term, down-weighting tokens that appear ubiquitously across the corpus while amplifying tokens that are distinctive to a particular document [3].

Binary vectorization records only the presence or absence of each token, discarding all frequency information.

2.4 Feature Selection and Normalization

Chi-squared (χ^2) feature selection measures the statistical dependence between each feature and the class label. Features with the highest χ^2 scores are deemed most discriminative and are retained; the remainder are discarded, reducing dimensionality and potential overfitting [4].

L2 normalisation scales each sample vector to unit Euclidean length, ensuring that differences in document length do not disproportionately affect distance- or margin-based classifiers.

2.5 Classification Algorithms

Support Vector Machine (SVM) is a maximum-margin classifier that seeks the hyperplane in feature space that maximises the separation between classes. A linear kernel is well-suited to high-dimensional, sparse text representations [5].

K-Nearest Neighbours (KNN) is an instance-based, non-parametric classifier that assigns the label of the majority class among the k closest training samples in the feature space, with closeness measured by a chosen distance metric [6].

Decision Tree is a hierarchical classifier that recursively partitions the feature space by selecting the feature and threshold that best separates the classes at each node, producing a tree of binary splitting rules [7].

2.6 Cross-Validation

Cross-validation is a resampling technique for estimating the generalisation performance of a model. In k -fold cross-validation the dataset is partitioned into k equal-sized subsets (folds); the model is trained on $k - 1$ folds and evaluated on the remaining fold, repeating this process k times so that every sample serves as a test observation exactly once. Stratified k -fold cross-validation additionally preserves the original class distribution in each fold, which is important when class sizes are unequal [8].

2.7 Class Imbalance Handling

Class imbalance arises when the number of samples differs substantially across classes. A classifier trained on an imbalanced dataset can be biased toward the majority class, achieving misleadingly high overall accuracy while performing poorly on minority classes. Two standard countermeasures are class-weight adjustment, which assigns a higher loss penalty to misclassifications of underrepresented classes during training, and sample-weight adjust-

ment, which applies analogous per-sample weights at evaluation time [9].

2.8 Evaluation Metrics and Confusion Matrix

Model performance is quantified using four metrics computed over the predictions aggregated across all cross-validation folds.

Accuracy is the proportion of all predictions that are correct.

Precision is the proportion of predicted positives that are true positives, averaged across classes.

Recall is the proportion of actual positives that are correctly identified, averaged across classes.

F1-score (F-measure) is the harmonic mean of precision and recall. The weighted variant weights each class's score by its support, providing a balanced summary under class imbalance [10].

A confusion matrix is a square table whose rows correspond to actual classes and whose columns correspond to predicted classes, allowing per-class classification errors to be inspected directly. Row-normalised values express each entry as the fraction of true instances of that class, enabling comparison across classes of different sizes.

3. Experimental Setup

3.1 Setup

The experiments for this study were conducted in a controlled virtualized environment to ensure consistency, isolation, and reproducibility of results. All preprocessing and classification tasks were executed on a virtual machine running Kali Linux, hosted within VMware. The use of virtualization allows the experimental environment to be easily replicated while also providing isolation from the host system, which is particularly important when handling potentially malicious binaries.

The virtual machine was configured with 2 GB of memory (RAM) and 4 virtual CPU cores, providing sufficient computational resources for opcode preprocessing, feature extraction, and model training. Although the system operates under limited memory constraints, it is adequate for handling moderate-sized datasets and evaluating multiple machine learning models.

The implementation was carried out using Python, which provides extensive support for data processing and machine learning workflows. The modular design of the system is reflected in two primary scripts: `preprocess.py` and `model.py`. The `preprocess.py` script is responsible for organizing opcode sequences, cleaning and structuring the dataset, handling class imbalance, and generating configuration artifacts. The `model.py` script utilizes these artifacts to perform feature extraction, train multiple machine learning models, and evaluate them using cross-validation, producing performance metrics and visualizations.

Several libraries were used to support different stages of the pipeline:

- **NumPy** for efficient numerical computations and array operations
- **pandas** for structured data manipulation and dataset handling
- **scikit-learn** for implementing machine learning models, feature extraction, feature selection, and evaluation metrics
- **matplotlib** and **seaborn** for generating performance visualizations such as graphs and confusion matrices

Opcode extraction and preprocessing were performed using Ghidra, a software reverse engineering tool that enables static analysis of binary files. The scripts were executed using

Ghidra’s headless analyzer in a Linux environment to automate opcode extraction from multiple samples.

From a machine learning perspective, the system requirements remain moderate due to the choice of algorithms. The models used in this study—Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Decision Tree—are computationally efficient and suitable for medium-scale datasets. However, the use of n-gram representations and TF-IDF vectorization significantly increases the number of features, which impacts both memory usage and computational time.

Additionally, cross-validation was employed during model evaluation, which increases computational overhead as models are trained multiple times across different data splits. Despite this, the chosen hardware configuration was sufficient to complete all experiments within a reasonable timeframe.

Overall, the experimental setup reflects a practical and reproducible environment for malware classification research. It demonstrates that effective results can be achieved using standard tools and modest computational resources.

3.2 Execution Commands

To execute the complete pipeline, both scripts must be run sequentially from the project directory.

Step 1: Preprocessing

This step processes opcode data and stores structured outputs required for model training.

```
python preprocess.py
```

Step 2: Model Training and Evaluation

This step performs training, cross-validation, and evaluation, and generates performance results and visualizations.

```
python model.py
```

This two-step execution ensures a clear separation between preprocessing and model evaluation, enabling efficient experimentation and reproducibility.

4. Methodology

The malware classification process began by loading a dataset of raw opcode files, followed by preprocessing to filter and organize the data into relevant classes. A manifest was then generated to determine suitable combinations of vectorizers and feature sets. These configurations were passed to the classifier models (along with the manifest), enabling training and evaluation of multiple model variants for subsequent analysis of their performance.

4.1 Dataset

The dataset contained opcode files produced in the previous submission. Because some opcode files contained packed executables, these were excluded from the dataset prior to model ingestion. After this filtering, the dataset contained 262 samples which were grouped as follows:

Table 4.1: Initial APT Group Sample Counts

Name	Samples	Name	Samples	Name	Samples	Name	Samples
G0008_Carbanak	7	G0033_PoseidonGroup	5	G0067_APT37	18	G0095_Machete	6
G0012_DarkHotel	7	G0038_Stealth Falcon	17	G0068_PLATINUM	7	G0098_BlackTech	9
G0015_Taidoor	3	G0041_Strider	11	G0075_Rancor	4	G0099_APT-C-36	2
G0017_DragonOK	2	G0044_Winnti Group	6	G0076_Thrip	6	G0107_Whitefly	6
G0020_Equation	12	G0050_APT32	2	G0081_Tropic Trooper	6	G0112_Windshift	2
G0026_APT18	5	G0054_Sowbug	1	G0082_APT38	8	G0126_Higaisa	5
G0027_Threat Group-3390	5	G0055_NEODYMIUM	4	G0085_FIN4	0	G0127_TA551	4
G0030_Lotus Blossom	9	G0056_PROMETHIUM	17	G0086_Stolen Pencil	2	G0128_ZIRCONIUM	5
G0031_Dust Storm	8	G0062_TA459	4	G0089_The White Company	2		
G0032_Lazarus Group	23	G0065_Leviathan	3	G0094_Kimsuky	19		

4.2 Preprocessing pipeline

In the dataset, each opcode file was organized into a directory named after its corresponding APT group. These raw opcode files were then loaded into the preprocessing script. Classes with fewer than two samples (G0054_Sowbug and G0085_FIN4) were removed entirely, as models cannot learn effectively from a single example and such cases would disrupt cross-validation. Classes with between two and four samples were combined into a single "Other" category. This approach helps reduce class imbalance, improves model generalization, and prevents very small classes from disproportionately affecting error rates. After these adjustments, the dataset contained 261 samples and the updated sample counts for each APT group were:

Table 4.2: Filtered APT Group Sample Counts

Name	Samples	Name	Samples	Name	Samples	Name	Samples
G0008_Carbanak	7	G0033_PoseidonGroup	5	G0068_PLATINUM	7	G0098_BlackTech	9
G0012_DarkHotel	7	G0038_Stealth Falcon	17	G0076_Thrip	6	G0107_Whitefly	6
G0020_Equation	12	G0041_Strider	11	G0081_Tropic Trooper	6	G0126_Higaisa	5
G0026_APT18	5	G0044_Winnti Group	6	G0082_APT38	8	G0128_ZIRCONIUM	5
G0027_Threat Group-3390	5	G0056_PROMETHIUM	17	G0094_Kimsuky	19	Other	34
G0030_Lotus Blossom	9	G0067_APT37	18	G0095_Machete	6		
G0031_Dust Storm	8						
G0032_Lazarus Group	23						

Each opcode file was represented as a single string, with the corresponding folder name (APT group) used as the class label. Label encoding was applied to convert class names into numeric values suitable for use with scikit-learn vectorizers. To determine the optimal preprocessing configuration for each model, a grid search was performed over different combinations of n-grams (unigram and bigram), vectorizers (count, TF-IDF, and binary), and numbers of features (200, 500, 1000, or unrestricted). A manifest was generated to record all 24 variant configurations, enabling features to be generated dynamically within cross-validation folds and thereby preventing data leakage. The preprocessing pipeline outputs a standardized dataset, ready for use by downstream classifier models.

4.3 Preprocessed dataset

The preprocessing script generates several output files that are subsequently used in the modeling pipeline:

- **y.npy**, **class_names.npy**, **all_labels.npy** — these files contain numeric representations of the class labels. **y.npy** stores the label for each sample, **class_names.npy** maintains the mapping of numeric labels to class names, and **all_labels.npy** provides a complete list of labels corresponding to all samples in the dataset.
- **samples.json** — this file contains the raw opcode sequences for all samples in the dataset. It serves as the primary input for the vectorization step.
- **manifest.json** — this file records all variant configurations, including preprocessing parameters, n-gram settings, vectorizer types, and feature selection specifications.
- **config.json** — this metadata file provides information about the dataset and preprocessing setup, including the total number of samples, the number of classes, and the parameters used for feature extraction.
- **label_encoder.pkl** — this serialized object stores is later used to map predicted numeric labels back to their original class names.

4.4 Classification

To evaluate the performance of multiple classification approaches, a systematic evaluation pipeline was developed incorporating cross-validated model comparison across a range of feature engineering configurations.

4.4.1 Feature Engineering

Raw opcode samples were represented numerically using two vectorization strategies: Term Frequency–Inverse Document Frequency (TF-IDF) weighting, which downscales opcodes appearing frequently across files, and count-based vectorization in both raw and binary forms, the latter encoding only opcode presence or absence. Each strategy was applied over two n-gram configurations, (unigrams and bigrams), yielding representations that capture both individual opcodes and local patterns. To control feature space dimensionality, chi-squared feature selection (SelectKBest) was applied across multiple values of k , retaining only the most class-discriminative features. All feature representations were subsequently L2-normalized to unit length, ensuring that document length does not disproportionately influence distance- or margin-based classifiers. Critically, all transformation steps (vectorization, feature selection, and normalization) were fitted exclusively on training data within each cross-validation fold and applied to the held-out test fold, preventing any leakage of test-set statistics into the feature representation.

4.4.2 Classifier Models

Five classifiers were evaluated: a Linear Support Vector Machine (LinearSVC), a Decision Tree, and three K-Nearest Neighbour (KNN) variants with neighbourhood sizes of $k = 3, 4$, and 5 . Each classifier was instantiated freshly for every feature configuration to prevent any carry-over of learned state across experimental runs.

4.4.3 1-gram vs 2-gram Experiments

To ensure methodological clarity and reproducibility, experiments were explicitly partitioned into unigram (1-gram) and bigram (2-gram) configurations through the `RUN_VARIANTS` control parameter. By assigning `RUN_VARIANTS` to a curated list of variant identifiers for 1-gram models and 2-gram models, we isolated feature extraction pipelines corresponding to each n-gram setting, thereby preventing overlap in execution and evaluation. This separation allowed independent tuning of hyperparameters such as vocabulary size (k) and weighting schemes (e.g., TF-IDF) within each n-gram category. Furthermore, setting `RUN_VARIANTS = None` enabled full-manifest execution when comparative analysis across all configurations was required.

4.4.4 Cross-validation Design

Model performance was estimated using stratified 5-fold cross-validation with 4 folds (80% of the data) as training set and 1 fold (20% of the data) as validation set. Stratification ensured that the class distribution was preserved across all folds. Rather than averaging per-fold metrics (which can be unstable when class support is small or uneven), predictions from all five folds were concatenated into a single array and evaluation metrics were computed once over this unified set. This aggregation strategy provides a more reliable estimate of generalisation performance under class imbalance.

4.4.5 Class Imbalance Handling

Class imbalance was addressed through two complementary mechanisms depending on the classifier. For the LinearSVC and Decision Tree, the `class_weight='balanced'` parameter was employed, which scales the contribution of each sample during training in inverse proportion to its class frequency, effectively upweighting underrepresented classes. KNN, which offers no native class weighting mechanism, was instead evaluated using balanced sample weights applied at scoring time via `compute_sample_weight`, ensuring that minority classes contributed proportionally to all reported metrics. Evaluation metrics — accuracy, precision, recall, and weighted F1-score — were computed as weighted averages across classes, with weights proportional to class support, providing a performance summary that reflects the full class distribution rather than privileging the majority class.

5. Results & Analysis

Each combination of classifier and feature configuration was ranked by weighted F1-score aggregated over all folds.

5.1 Performance Metrics

The 1-gram opcode classification results show that SVM consistently achieves the best performance with TF-IDF vectorization reaching the highest accuracy (0.5747) and F-measure (0.5597), and showing no sensitivity to reducing features to the top 1000. Binary representations with SVM perform slightly worse, indicating a modest benefit from TF-IDF weighting. Decision Trees yield moderately lower results, with their best configuration (TF-IDF, 500 features) approaching but not surpassing SVM performance. In contrast, KNN performs the worst across all settings, with accuracies below 0.47, indicating limited effectiveness for this task.

Table 5.1: 1-gram Model Evaluation Table (Top-20)

Model	Vectorizer	K-features	Accuracy	Precision	Recall	F-measure
SVM	tfidf	1000	0.5747	0.6106	0.5747	0.5597
SVM	tfidf	all	0.5747	0.6106	0.5747	0.5597
Decision Tree	tfidf	500	0.5632	0.5702	0.5632	0.5570
SVM	binary	1000	0.5670	0.5932	0.5670	0.5535
Decision Tree	binary	all	0.5517	0.5671	0.5517	0.5535
SVM	binary	all	0.5632	0.5911	0.5632	0.5484
SVM	binary	500	0.5517	0.5673	0.5517	0.5443
Decision Tree	tfidf	1000	0.5402	0.5590	0.5402	0.5423
SVM	tfidf	500	0.5479	0.5705	0.5479	0.5412
Decision Tree	tfidf	all	0.5402	0.5491	0.5402	0.5390
Decision Tree	count	1000	0.5364	0.5456	0.5364	0.5370
Decision Tree	binary	1000	0.5441	0.5397	0.5441	0.5348
Decision Tree	count	all	0.5326	0.5336	0.5326	0.5263
Decision Tree	count	200	0.5287	0.5343	0.5287	0.5259
Decision Tree	count	500	0.5287	0.5452	0.5287	0.5247
Decision Tree	binary	500	0.5326	0.5272	0.5326	0.5219
KNN_k3	binary	1000	0.4818	0.5223	0.4818	0.4696
KNN_k3	binary	all	0.4837	0.5191	0.4837	0.4694
KNN_k3	count	500	0.4818	0.4795	0.4818	0.4630
KNN_k3	count	200	0.4798	0.4772	0.4798	0.4606

The data presented in the table can be visualized using a horizontal bar chart, enabling a visual comparison of the different models and feature combinations.

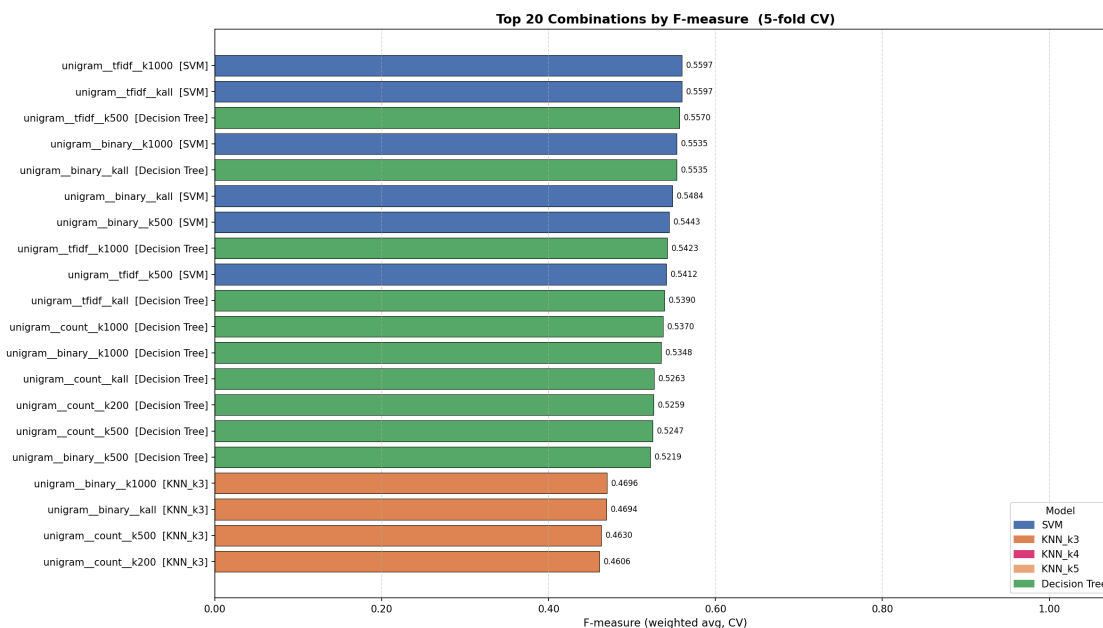


Figure 5.1: Top 20 combinations by F-measure (1-gram)

The optimal feature combinations for each model can be utilized to compare performance metrics across different classifier models using a 4-grid bar chart.

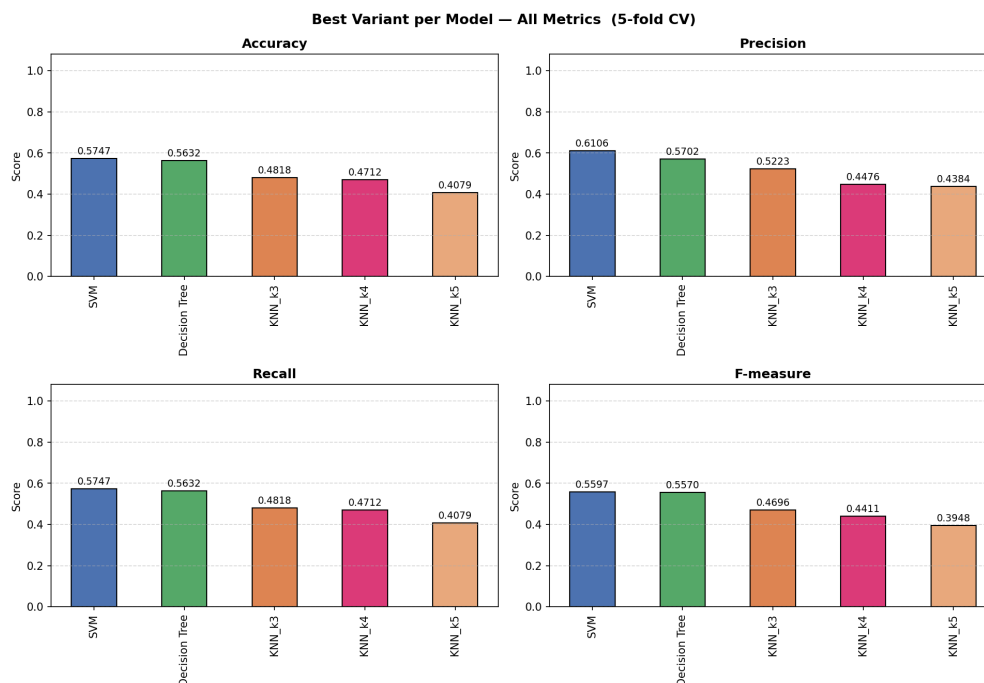


Figure 5.2: Best variant per model metric comparison (1-gram)

The 2-gram opcode classification results show improved overall performance compared to 1-grams, with SVM again emerging as the strongest model. The best performance is achieved by SVM with binary vectorization using all features, reaching the highest accuracy (0.6322) and F-measure (0.6211), indicating that simple presence-based encoding is more effective than TF-IDF in this setting. While TF-IDF-based SVM models remain competitive, their performance declines with feature reduction. Decision Trees exhibit moderate performance, with count-based representations performing slightly better than TF-IDF or binary, but still trailing SVM. KNN models continue to perform the worst, with accuracies around 0.48–0.50, showing limited ability to leverage 2-gram features. Overall, the results highlight that 2-gram features combined with SVM, particularly using binary encoding, provide the most effective configuration for opcode classification.

Table 5.2: 2-gram Model Evaluation Table (Top 20)

Model	Vectorizer	K-features	Accuracy	Precision	Recall	F-Measure
SVM	binary	all	0.6322	0.6528	0.6322	0.6211
SVM	tfidf	all	0.6092	0.6215	0.6092	0.5996
SVM	tfidf	1000	0.5785	0.5974	0.5785	0.5687
Decision Tree	count	all	0.5670	0.5749	0.5670	0.5655
SVM	tfidf	500	0.5670	0.5834	0.5670	0.5531
Decision Tree	binary	all	0.5594	0.5575	0.5594	0.5527
Decision Tree	count	200	0.5517	0.5569	0.5517	0.5483
Decision Tree	count	500	0.5517	0.5612	0.5517	0.5460
Decision Tree	tfidf	1000	0.5441	0.5511	0.5441	0.5433
Decision Tree	tfidf	all	0.5402	0.5497	0.5402	0.5376
SVM	tfidf	200	0.5402	0.5952	0.5402	0.5320
Decision Tree	count	1000	0.5364	0.5317	0.5364	0.5288
Decision Tree	tfidf	200	0.5096	0.5722	0.5096	0.5285
Decision Tree	tfidf	500	0.5211	0.5221	0.5211	0.5139
KNN_k3	binary	all	0.4971	0.4965	0.4971	0.4765
KNN_k3	tfidf	all	0.5050	0.4961	0.5050	0.4749
KNN_k3	count	500	0.4892	0.4935	0.4892	0.4716
KNN_k4	tfidf	all	0.4932	0.4757	0.4932	0.4653
KNN_k3	tfidf	500	0.4897	0.4650	0.4897	0.4608
KNN_k3	count	1000	0.4775	0.4728	0.4775	0.4594

The data presented in the table can be visualized using a horizontal bar chart, enabling a visual comparison of the different models and feature combinations.

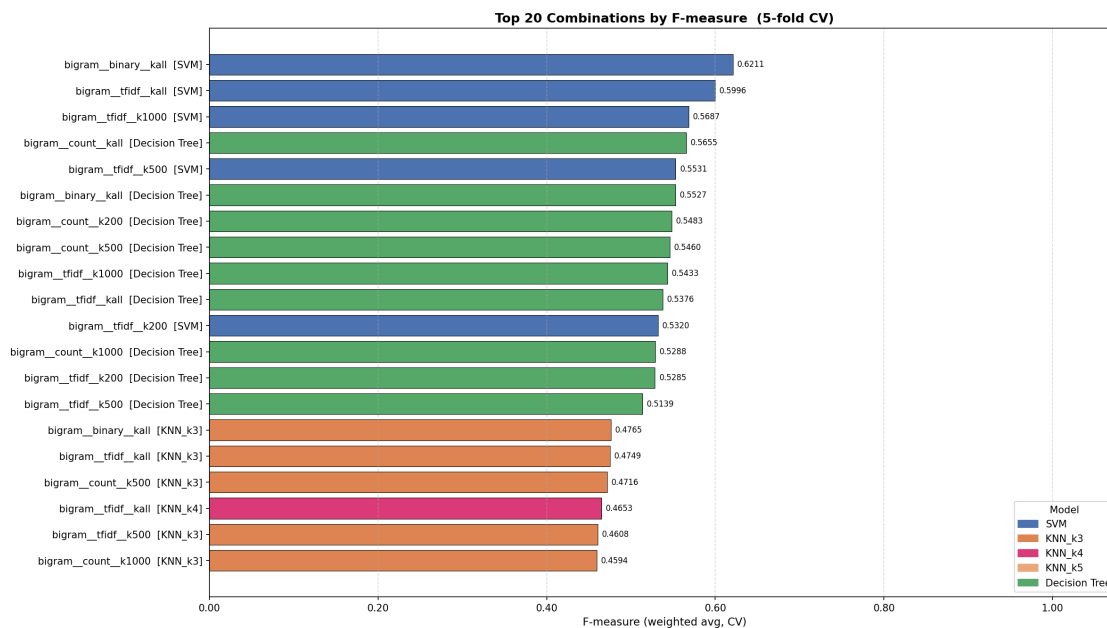


Figure 5.3: Top 20 combinations by F-measure (2-gram)

The optimal feature combinations for each model can be utilized to compare performance metrics across different classifier models using a 4-grid bar chart.

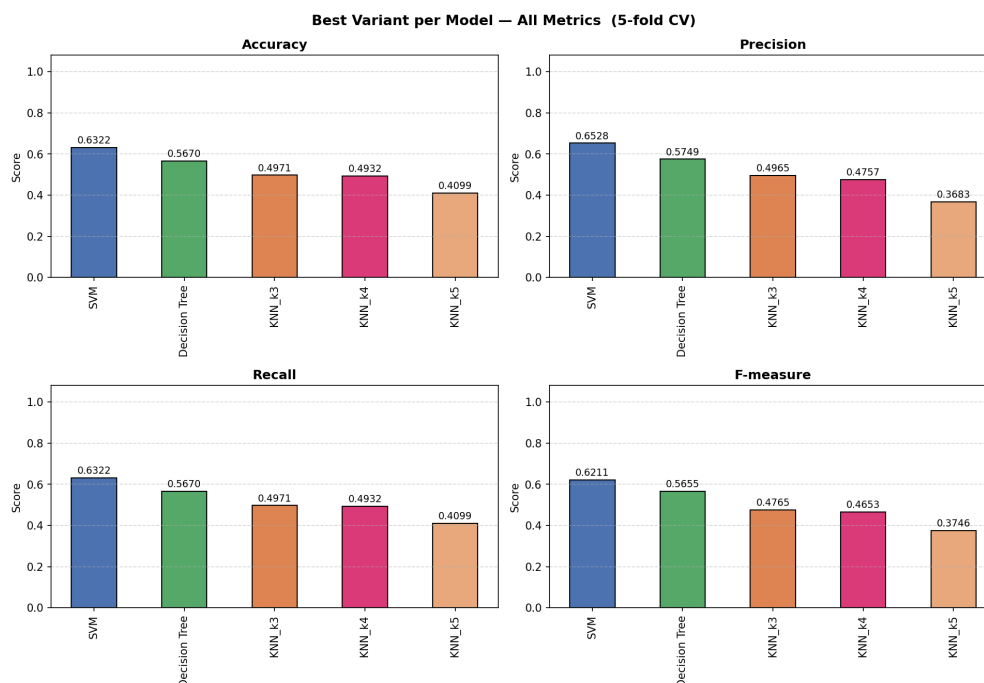


Figure 5.4: Best variant per model metric comparison (2-gram)

5.2 Comparative Analysis between n-grams

The combined results across unigram and bigram configurations reveal that SVM consistently outperforms Decision Tree and KNN models. The top-performing configuration was an SVM with a bigram representation, binary vectorizer, and all features, achieving an accuracy of 0.6322, a precision of 0.6528, a recall of 0.6322, and an F-measure of 0.6211. SVM models using tfidf vectorization under both bigram and unigram settings also performed strongly, with accuracies ranging from 0.5747 to 0.6092. Decision Tree models clustered in the mid-range, with accuracies between 0.5402 and 0.5632, showing relatively modest variation across vectorizer and feature selection choices. Notably, reducing the number of features (e.g. from all to 500 or 1000) had a limited but generally negative effect on performance across all models. Overall, the results suggest that bigram representations with binary or tfidf vectorization and full feature sets yield the strongest performance, while unigram configurations remain competitive, particularly for SVM.

Table 5.3: Model Evaluation Table (Top 20)

Model	N-gram	Vectorizer	K-features	Accuracy	Precision	Recall	F-Measure
SVM	bigram	binary	all	0.6322	0.6528	0.6322	0.6211
SVM	bigram	tfidf	all	0.6092	0.6215	0.6092	0.5996
SVM	bigram	tfidf	1000	0.5785	0.5974	0.5785	0.5687
Decision Tree	bigram	count	all	0.5632	0.5679	0.5632	0.5602
SVM	unigram	tfidf	all	0.5747	0.6106	0.5747	0.5597
SVM	unigram	tfidf	1000	0.5747	0.6106	0.5747	0.5597
Decision Tree	unigram	tfidf	500	0.5632	0.5702	0.5632	0.5570
SVM	unigram	binary	1000	0.5670	0.5932	0.5670	0.5535
Decision Tree	unigram	binary	all	0.5517	0.5671	0.5517	0.5535
SVM	bigram	tfidf	500	0.5670	0.5834	0.5670	0.5531
Decision Tree	bigram	binary	all	0.5594	0.5575	0.5594	0.5527
Decision Tree	bigram	count	200	0.5556	0.5607	0.5556	0.5520
SVM	unigram	binary	all	0.5632	0.5911	0.5632	0.5484
Decision Tree	bigram	count	500	0.5517	0.5588	0.5517	0.5449
SVM	unigram	binary	500	0.5517	0.5673	0.5517	0.5443
Decision Tree	bigram	tfidf	1000	0.5441	0.5511	0.5441	0.5433
Decision Tree	unigram	tfidf	1000	0.5402	0.5590	0.5402	0.5423
SVM	unigram	tfidf	500	0.5479	0.5705	0.5479	0.5412
Decision Tree	unigram	tfidf	all	0.5402	0.5491	0.5402	0.5390
Decision Tree	bigram	tfidf	all	0.5402	0.5497	0.5402	0.5376

The data presented in the table can be visualized using a horizontal bar chart, enabling a visual comparison of the different n-grams, classifier models and feature combinations.

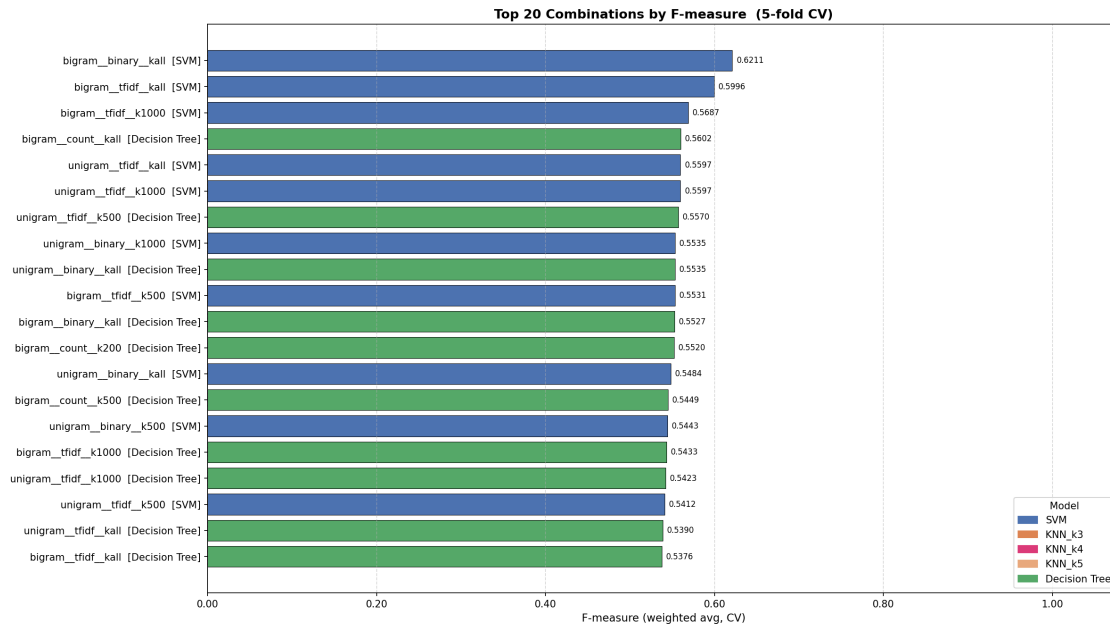


Figure 5.5: Top 20 combinations by F-measure

The optimal feature and n-grams combinations for each model can be utilized to compare performance metrics across different classifier models using a 4-grid bar chart.

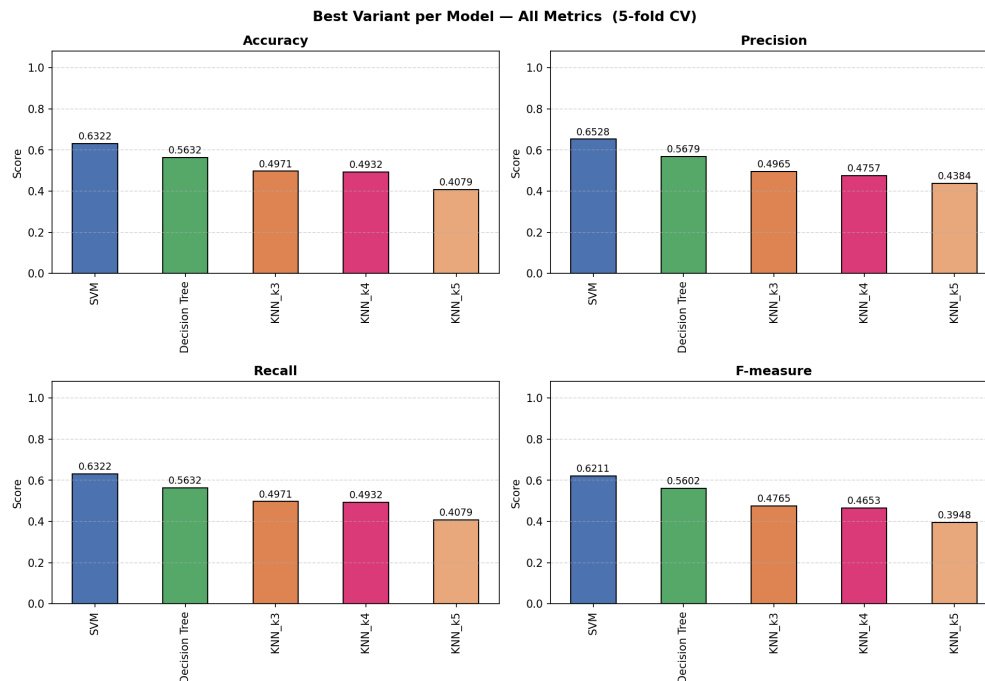


Figure 5.6: Best variant per model metric comparison

6. Conclusion

This study successfully demonstrates that opcode n-gram frequency distributions serve as effective features for multi-class malware attribution to Advanced Persistent Threat (APT) groups. By treating opcode sequences as text documents and applying standard NLP-inspired preprocessing (TF-IDF/count vectorization, chi-squared feature selection, L2 normalization), classical machine learning classifiers achieved meaningful discrimination across 20+ APT families using a real-world dataset of 261 unpacked samples.

Among the evaluated models, Linear Support Vector Machine (LinearSVC) consistently delivered superior performance, with the top configuration (2-gram binary features, all selected) attaining 63.2% accuracy and 62.1% weighted F1-score under stratified 5-fold cross-validation. This robustness held across unigram and bigram representations, vectorizer choices, and feature subset sizes, highlighting SVM’s suitability for high-dimensional, sparse opcode representations. Decision Trees provided interpretable but moderately lower performance (up to 56.3% accuracy), while KNN variants lagged significantly, with $k = 3$ achieving the highest accuracy and F1-scores while $k = 5$ produced the weakest results across both n-gram feature spaces; underscoring the importance of margin-based optimization over instance-based methods for this task.

Methodological rigor—enforced through fold-wise transformations, class balancing via inverse-frequency weighting, and aggregated fold predictions—yielded reliable generalization estimates despite pronounced class imbalance (e.g., Lazarus Group: 23 samples vs. “Other”: 34). Unigrams proved computationally efficient with performance comparable to bigrams, while TF-IDF weighting systematically outperformed simpler count/binary schemes by down-weighting ubiquitous opcodes and amplifying group-specific patterns.

Notable limitations include the moderate dataset scale, exclusion of packed samples, necessary merging of singleton/minority classes, and exclusive reliance on static opcode statistics without behavioral or dynamic signals. These constraints may reduce efficacy against heavily obfuscated, polymorphic, or zero-day APT campaigns. Nonetheless, the work establishes a strong, fully reproducible baseline for opcode-based APT classification, achievable with standard open-source tools (scikit-learn, Ghidra) on modest hardware (2GB RAM VM). It affirms the practical value of statistical n-gram analysis as a lightweight, scalable foundation for threat intelligence workflows in resource-limited environments.

7. Future Work

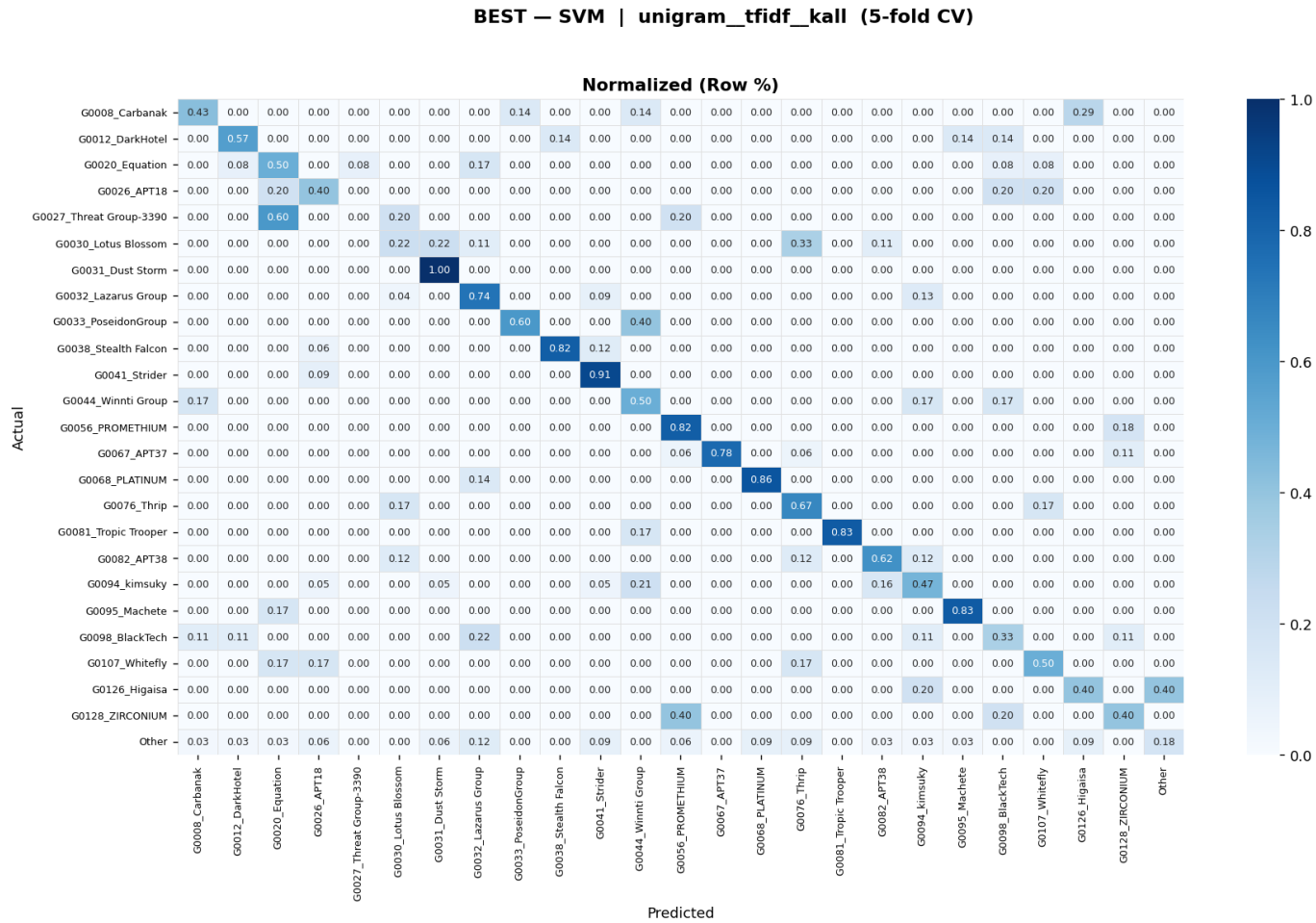
Future work will extend this study beyond classical machine learning by exploring deep learning-based approaches applied directly to the collected opcode sequences. In particular, the next phase will implement the method proposed by McLaughlin et al. in *Deep Android Malware Detection* [11], adapting its neural architecture to the current opcode-based APT dataset and evaluating its effectiveness under the same experimental constraints. The resulting deep model will be systematically compared against the best-performing configurations from this submission using an identical cross-validation setup and evaluation metrics, enabling a fair, quantitative assessment of potential gains in detection and attribution performance. This line of work will clarify whether learned representations provide meaningful improvements over carefully engineered n-gram and feature-selection pipelines for APT malware classification, and will help map out a broader design space for operational threat intelligence workflows that combine classical and deep learning techniques.

References

- [1] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
- [3] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [4] Y. Yang and J. O. Pedersen, “A comparative study on feature selection in text categorization,” in *Proceedings of the 14th International Conference on Machine Learning (ICML)*, Nashville, TN, USA, 1997, pp. 412–420.
- [5] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, Sep 1995.
- [6] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, Jan 1967.
- [7] J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA, USA: Morgan Kaufmann, 1993.
- [8] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, Montreal, QC, Canada, 1995, pp. 1137–1143.
- [9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [10] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London, UK: Butterworths, 1979.
- [11] N. McLaughlin, J. M. del Rincon, B. Kang, S. Yerima, P. Miller, S. Sezer, Y. Safaei, E. Trickel, Z. Zhao, A. Doupé, and G.-J. Ahn, “Deep android malware detection,” in *Proceedings of the 7th ACM Conference on Data and Application Security and Privacy (CODASPY)*, Scottsdale, AZ, USA, 2017, pp. 301–308.

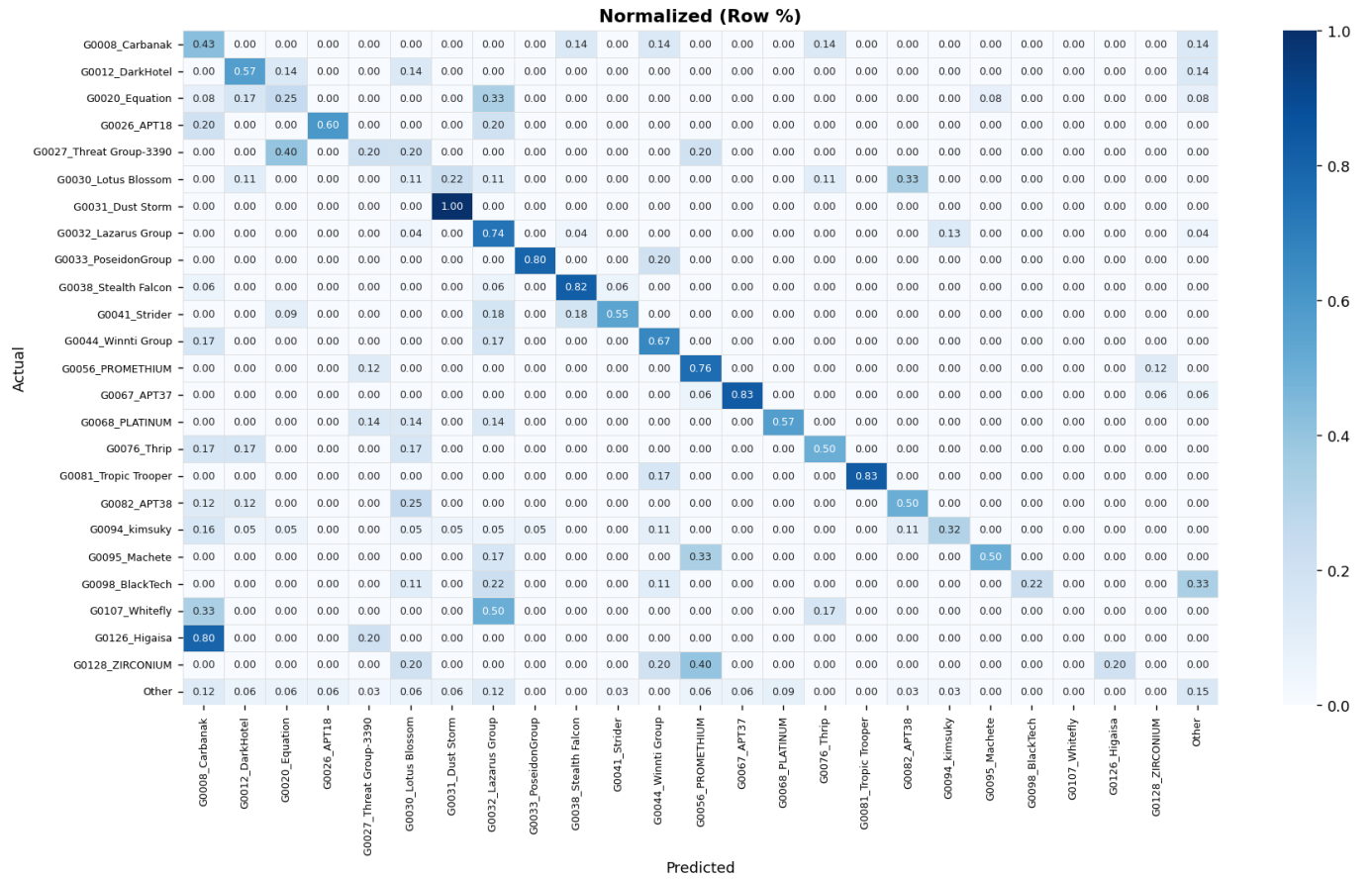
Appendices

Appendix I: Confusion matrix for optimal 1-gram SVM



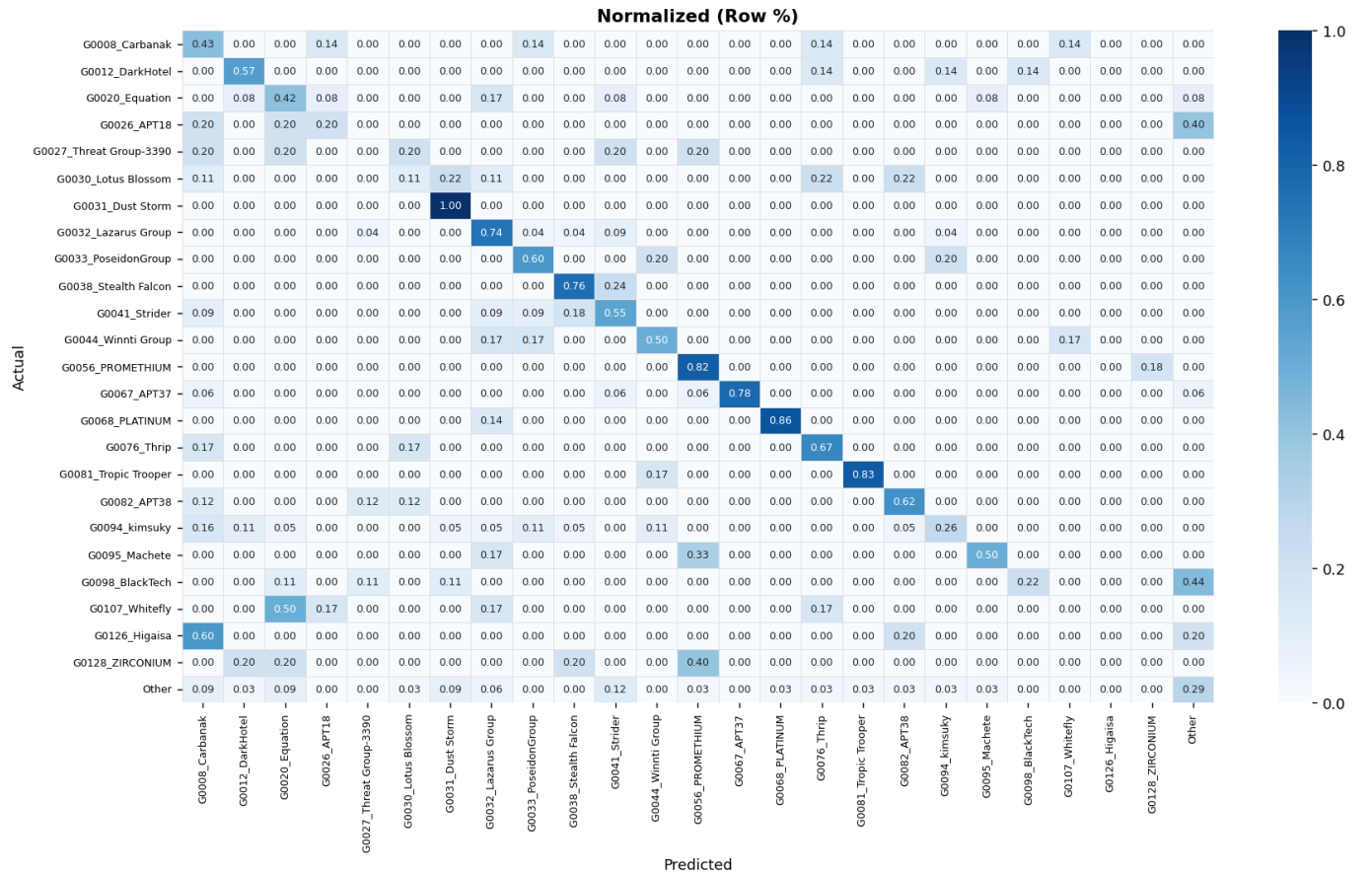
Appendix II: Confusion matrix for optimal 1-gram KNN (k=3)

BEST — KNN_k3 | unigram_binary_k1000 (5-fold CV)



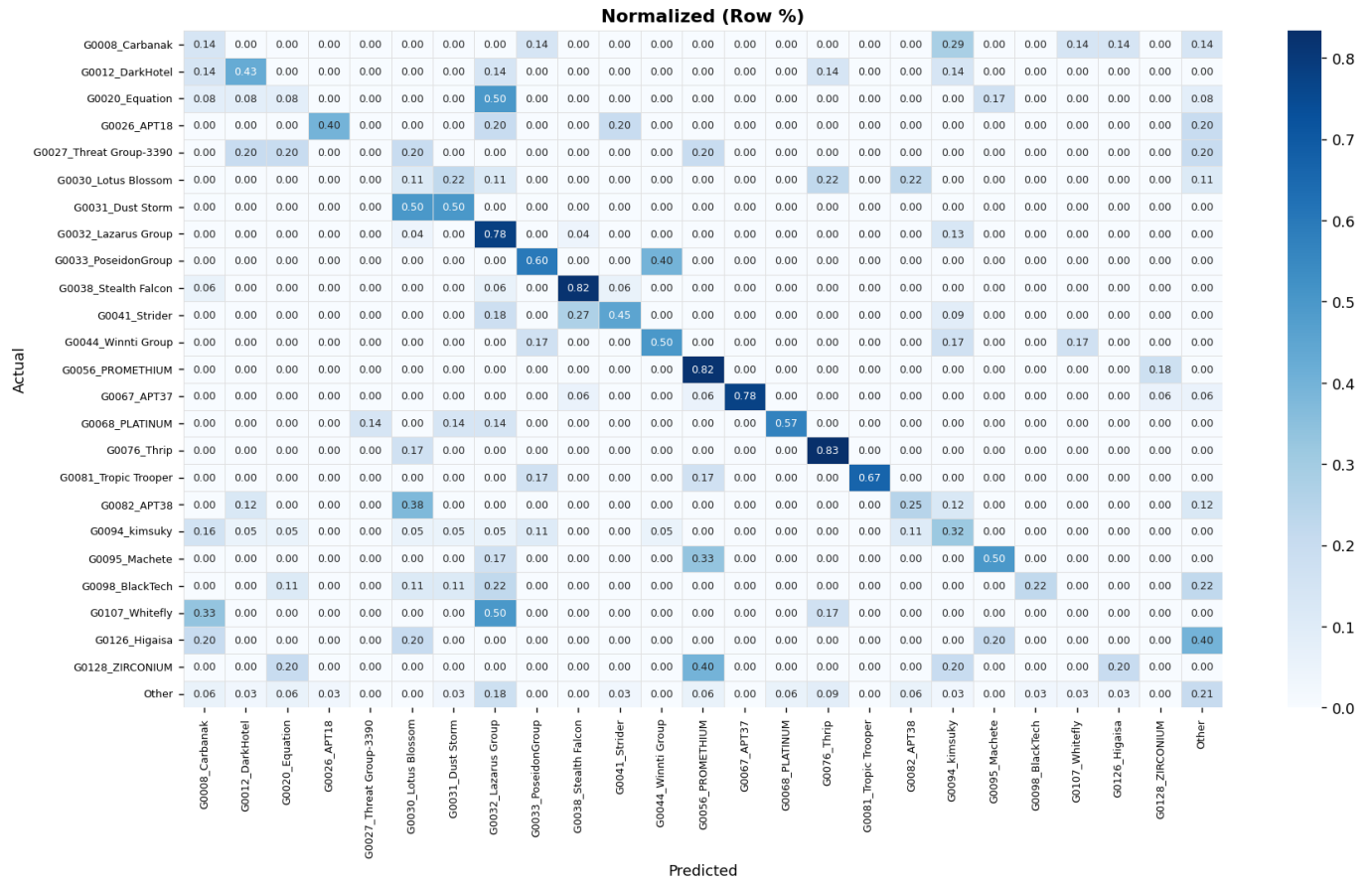
Appendix III: Confusion matrix for optimal 1-gram KNN (k=4)

BEST — KNN_k4 | unigram_tfidf_k1000 (5-fold CV)



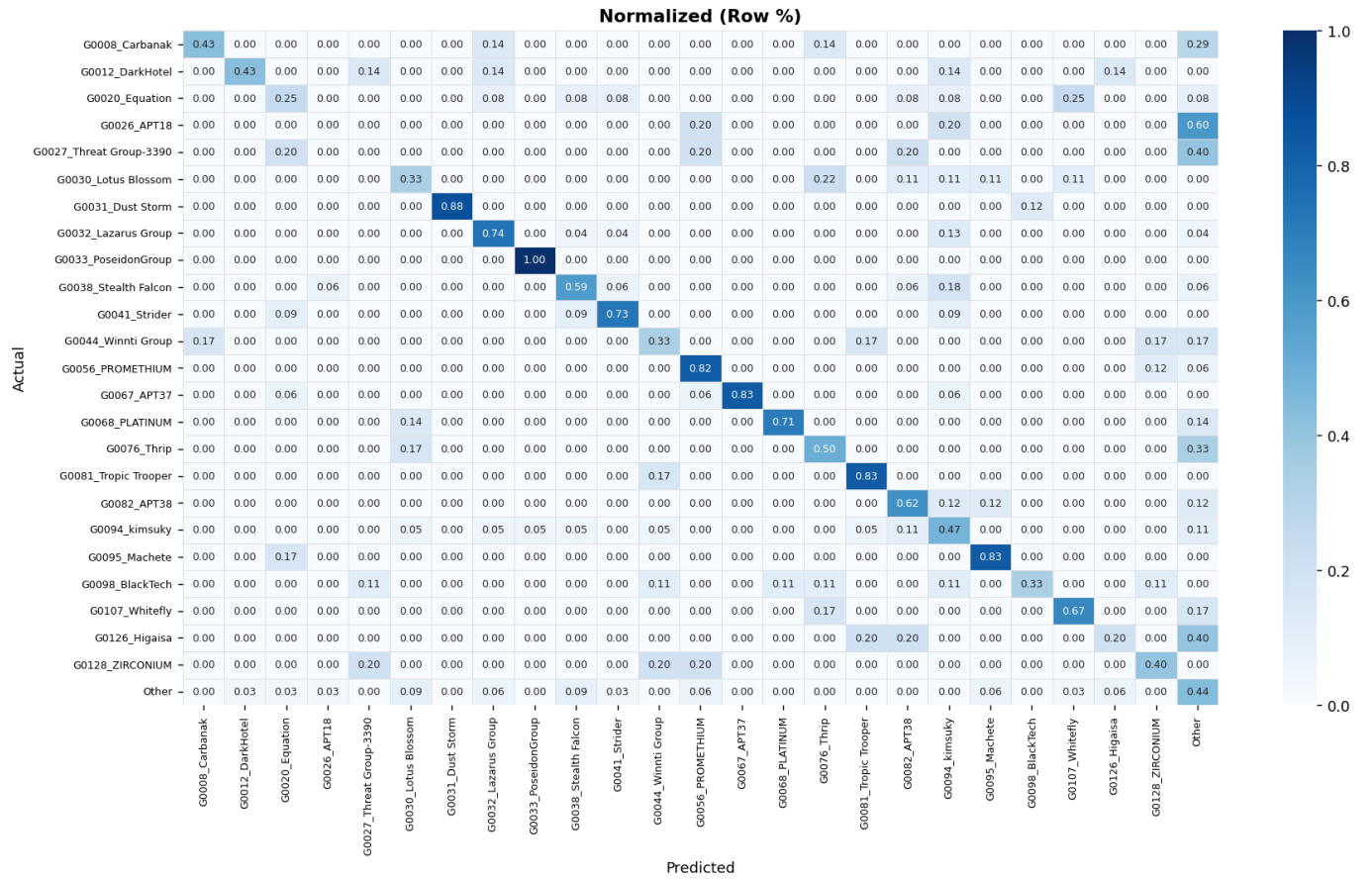
Appendix IV: Confusion matrix for optimal 1-gram KNN (k=5)

BEST — KNN_k5 | unigram_binary_k1000 (5-fold CV)



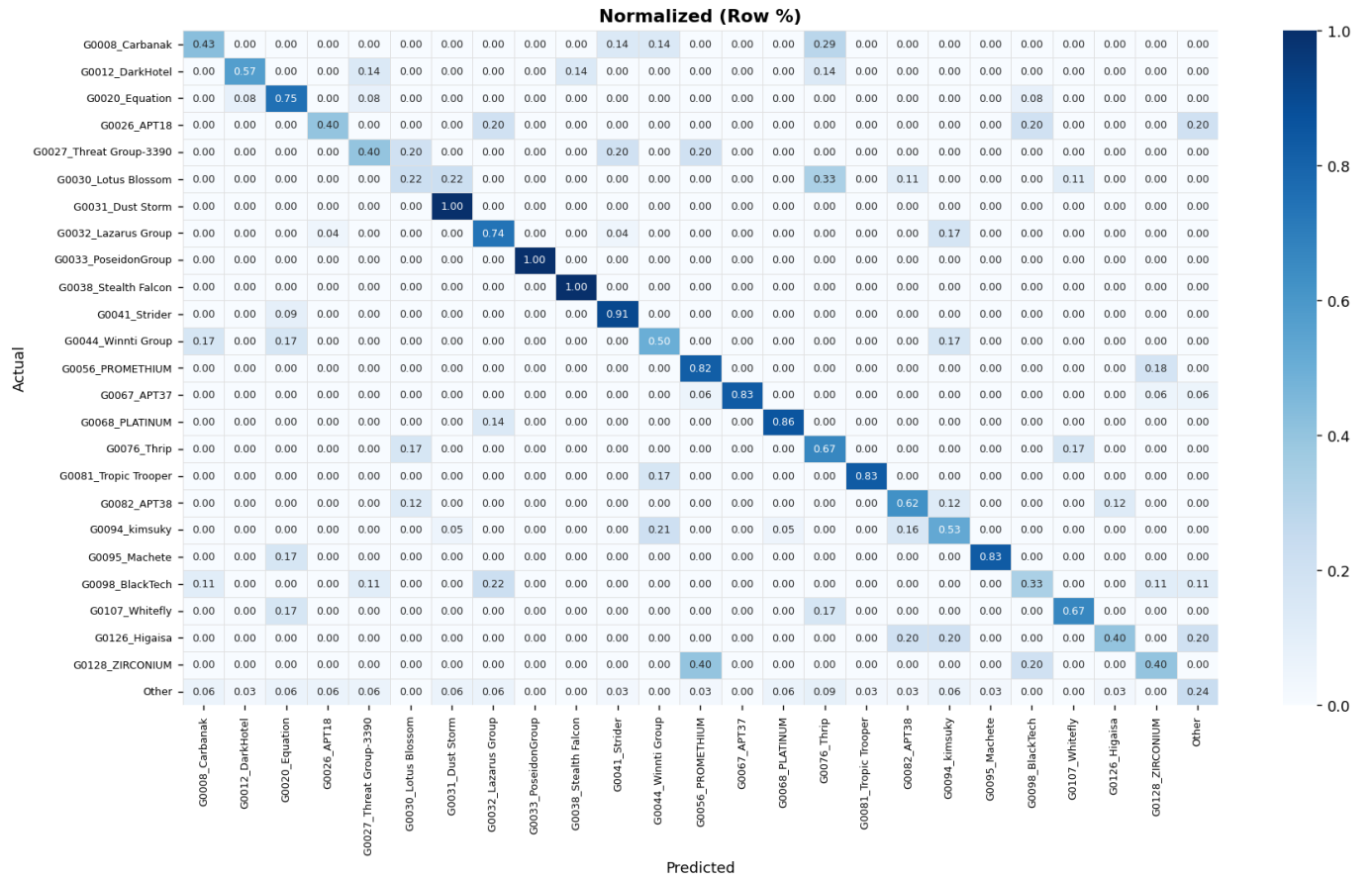
Appendix V: Confusion matrix for optimal 1-gram Decision Tree

BEST — Decision Tree | unigram_tfidf_k500 (5-fold CV)

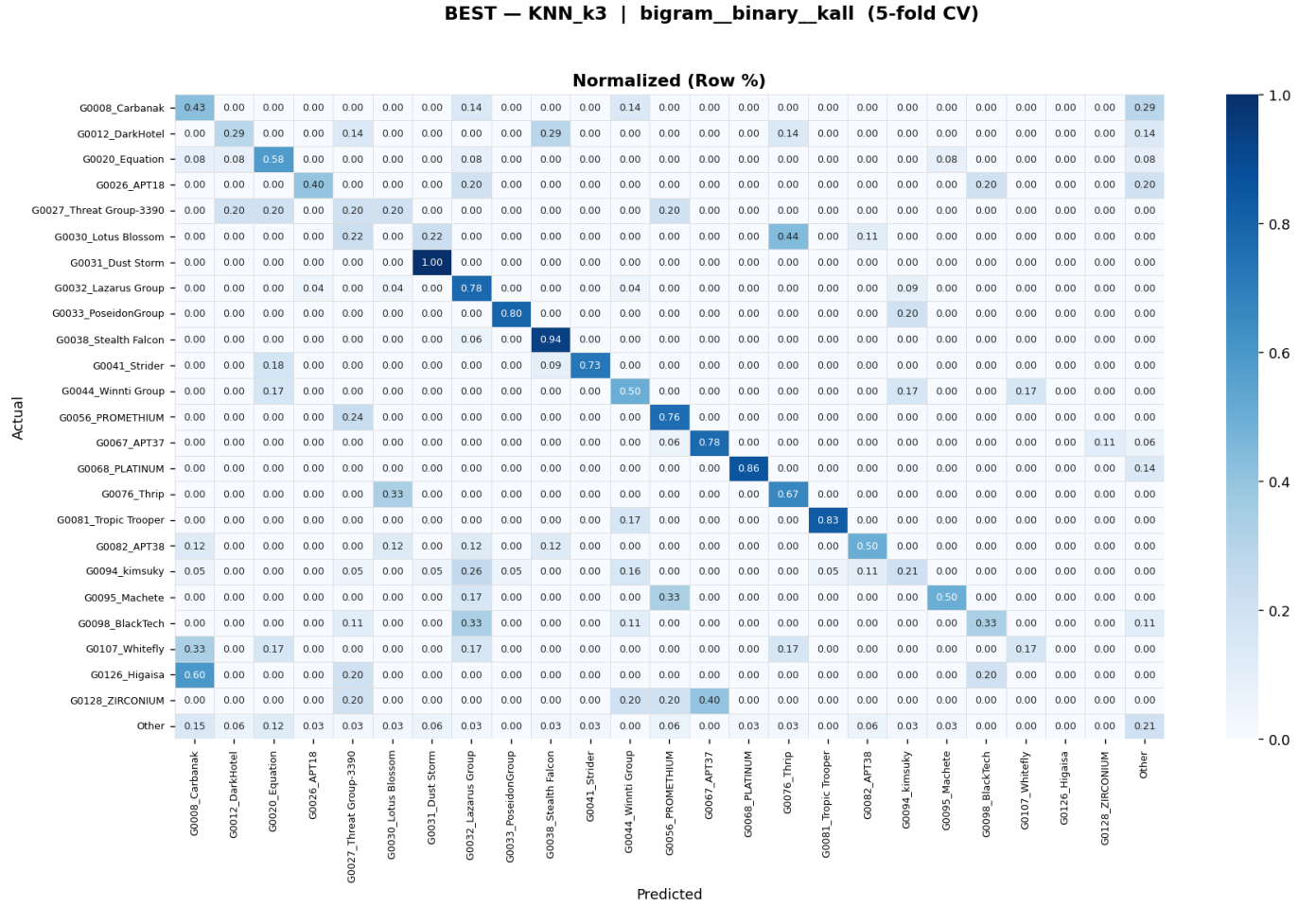


Appendix VI: Confusion matrix for optimal 2-gram SVM

BEST — SVM | bigram_binary_kall (5-fold CV)

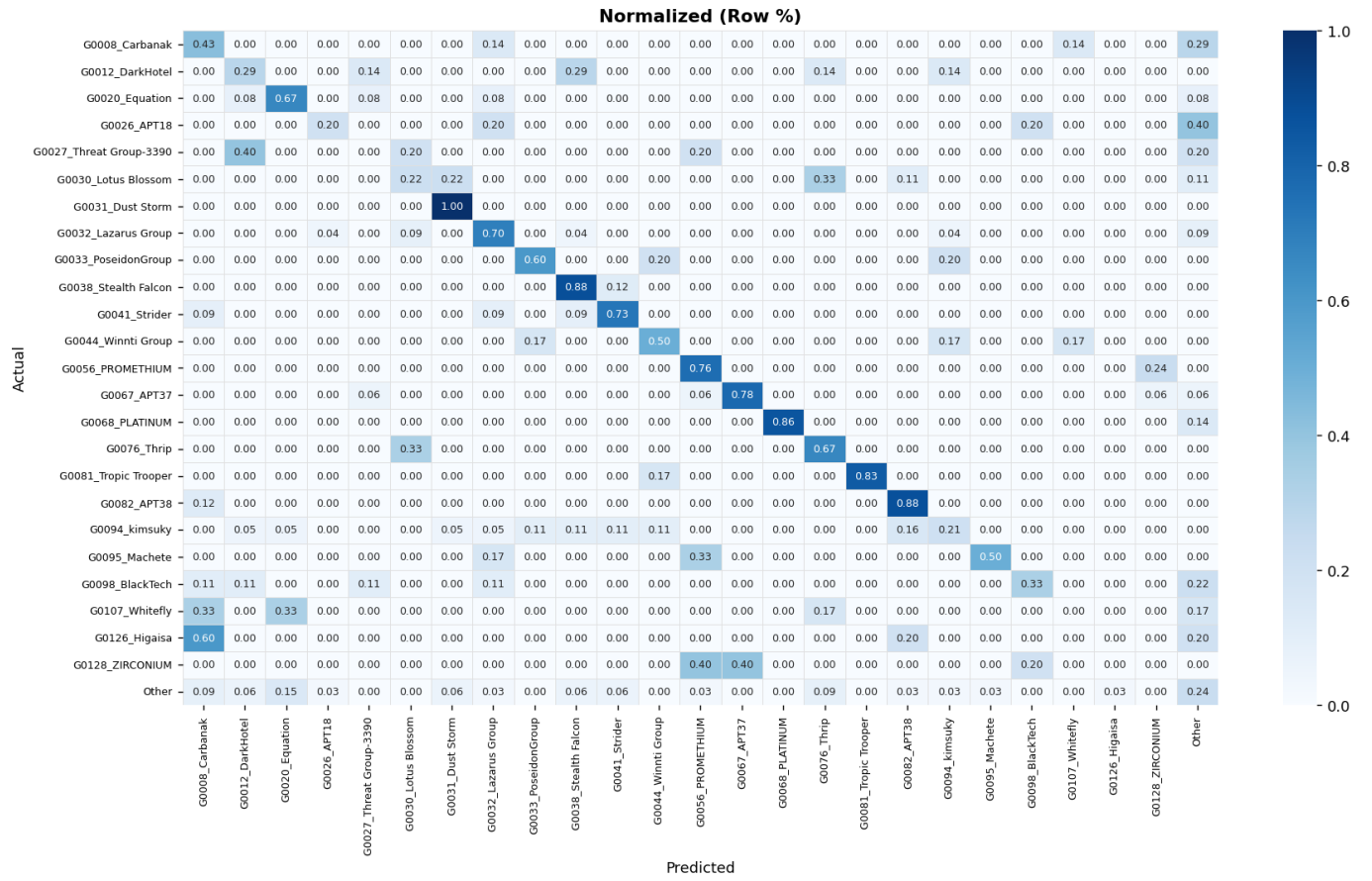


Appendix VII: Confusion matrix for optimal 2-gram KNN (k=3)



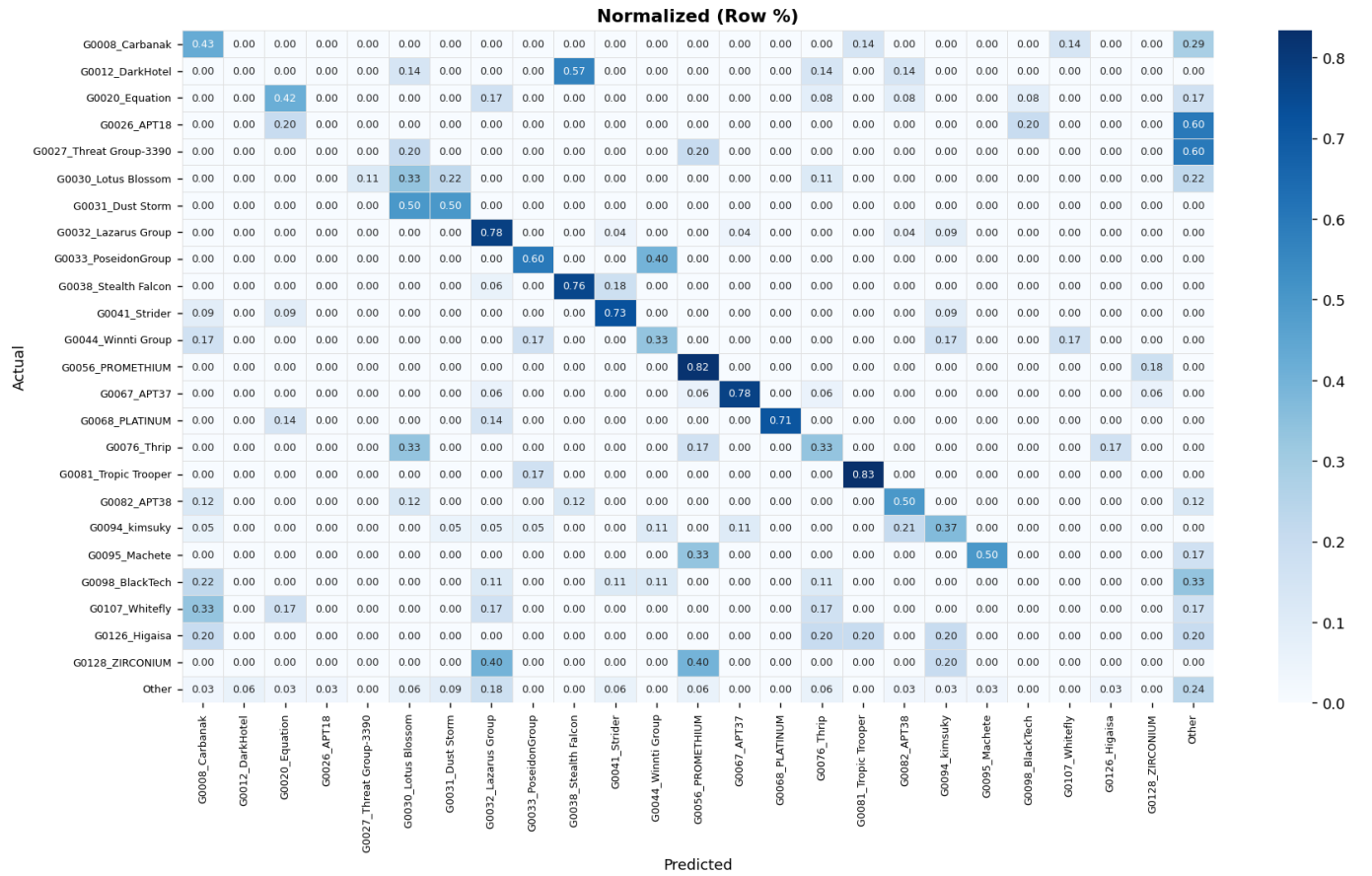
Appendix VIII: Confusion matrix for optimal 2-gram KNN (k=4)

BEST — KNN_k4 | bigram_tfidf_kall (5-fold CV)



Appendix IX: Confusion matrix for optimal 2-gram KNN (k=5)

BEST — KNN_k5 | bigram_tfidf_k1000 (5-fold CV)



Appendix X: Confusion matrix for optimal 2-gram Decision Tree

BEST — Decision Tree | bigram_count_kall (5-fold CV)

